

Ranking Android Development Languages by Energy Efficiency

Naya Al Afifi
Computer Science, NYUAD
na3266@nyu.edu

Advised by: João Paulo Fernandes, Karim Ali

ABSTRACT

Modern Android application development supports several programming language options, including Java, Kotlin, C, C++, and embedded JavaScript engines. Each of these languages interacts differently with Android's runtime, compiler pipelines, and hardware subsystems, which may influence execution time and energy consumption in ways that directly affect battery life. Given that mobile devices operate under strict battery constraints and that sustainability concerns continue to grow, understanding whether language choice impacts energy efficiency is an important and practical issue for both developers and organizations.

This Capstone project investigates three central research questions: (1) Does programming language choice significantly affect runtime under identical computational workloads? (2) Do specific languages perform better for CPU-bound tasks versus memory-bound tasks on Android? (3) Is there a consistent performance and energy ranking across languages? Prior research on general-purpose programming languages has shown that compiled languages often outperform interpreted or virtual machine-based languages across metrics such as execution time, energy usage, and memory footprint. However, these studies were not conducted on Android, whose execution model relies on a unique runtime environment (ART), a hybrid JIT/AOT compilation strategy, and system behaviors specific to mobile hardware. Similarly, while foundational computational benchmarks exist for Android, empirical research that examines energy efficiency across multiple Android-supported languages using

standardized benchmark workloads and modern on-device tracing tools remains scarce.

This project addresses this gap by conducting a systematic, controlled comparison of five languages commonly used for performance-critical computations on Android: Java, Kotlin, C, C++, and JavaScript.

KEYWORDS

Android, Energy Efficiency, Programming Languages, Benchmarking, Performance Analysis, Mobile Computing, Green Software

Reference Format:

Naya Al Afifi. 2025. Ranking Android Development Languages by Energy Efficiency. In *NYUAD Capstone Project 1 Reports, Fall 2025, Abu Dhabi, UAE*. 10 pages.

1 INTRODUCTION

Mobile devices, especially Android smartphones, are ubiquitous computing platforms operating under the significant constraint of limited battery capacity. As a result, application energy consumption remains a crucial factor influencing user experience, device usability, and the broader sustainability implications associated with frequent charging. Poorly optimized applications can cause rapid battery drain, generate user dissatisfaction, and negatively affect adoption. For developers, this raises an important question: how much does the choice of programming language affect performance and, by extension, estimated energy usage on Android devices?

The Android ecosystem provides several distinct programming languages for application and computational development. Java remains widely used due to its long history on the platform, while Kotlin has become Google's preferred language and is now the default option for new Android development. For performance-critical tasks, developers may turn to C or C++ through the Android NDK to obtain closer-to-hardware execution. Additionally, JavaScript executed via the Jetpack JavaScriptEngine enables computational scripting within native applications. Although each language involves different runtime behaviors, compilation modes, and

This report is submitted to NYUAD's capstone repository in fulfillment of NYUAD's Computer Science major graduation requirements.

جامعة نيويورك أبوظبي



Capstone Project 1, Fall 2025, Abu Dhabi, UAE
© 2025 New York University Abu Dhabi.

system interactions, comprehensive empirical data comparing them under standardized conditions on actual Android devices remains limited.

Developers often choose a language for reasons such as ecosystem maturity, readability, safety features, or tooling, but the energy implications of this choice tend to be less understood. This lack of visibility is due to several factors: energy profiling on mobile devices requires specialized tools, device conditions can vary significantly, and the interactions between different runtimes (e.g., ART for Java/Kotlin versus native execution for NDK code) make direct comparisons non-trivial. However, understanding these differences is increasingly important for designing sustainable applications and optimizing resource-intensive workloads.

To address these concerns, this study is guided by three central research questions:

- (1) **RQ1:** Does programming language choice significantly affect runtime under identical computational workloads?
- (2) **RQ2:** Do specific languages perform better for CPU-bound tasks versus memory-bound tasks on Android?
- (3) **RQ3:** Is there a consistent performance and energy ranking across languages?

This research fills that gap by focusing strictly on computational microbenchmarks and analyzing their behavior across languages under controlled conditions. The aim is not to compare UI frameworks, cross-platform toolkits, or networked application performance, but instead to isolate pure computation-level differences tied directly to the language implementation. This narrower focus enables controlled comparisons and more reliable interpretations of how language choice interacts with CPU scheduling, instruction throughput, runtime optimizations, and subsystem activity that may correlate with energy usage.

2 RELATED WORK

Several studies have investigated energy consumption in software systems and mobile applications, forming the foundation for this work. Pereira et al. [5] conducted a well-known comparative study across multiple general-purpose programming languages, examining execution time, memory usage, and energy efficiency. Their results showed that compiled languages typically outperform interpreted or virtual machine-based ones, offering a valuable general-purpose baseline for understanding language-level performance characteristics. However, because these comparisons were conducted on desktop or server environments, they do not capture the constraints, optimizations, and runtime behaviors unique to Android's mobile execution model. As a result, while informative, these findings cannot be directly applied to Android development without further platform-specific investigation.

Research explicitly focused on Android has also emerged. Chen and Zong [1] studied how language choice, runtime configuration, and compiler design influence energy efficiency in Android applications. Their work highlights the complexity introduced by Android Runtime (ART), JIT and AOT compilation, and mobile-specific system services. Similarly, Oliveira et al. have produced several studies relevant to understanding Android performance, including analyses of energy consumption across different app development approaches [4] and examinations of runtime overhead in mobile application environments [3]. These studies reinforce that Android's execution environment introduces layers of variability, such as runtime warm-up, scheduling decisions, and thermal limitations, that significantly affect performance and energy characteristics.

A particularly relevant contribution is the Energy-Benchmark-Languages-Android repository created by the Green Software Lab [2]. This repository provides Android implementations of benchmarks from The Computer Language Benchmarks Game, covering 28 languages across 10 distinct benchmarks. The repository includes a RAPL-based energy measurement framework and compilation scripts for desktop/server environments. This study adopts six benchmark specifications from this repository (BinaryTrees, FannkuchRedux, Fasta, Mandelbrot, RevComp, SpectralNorm) but employs an entirely distinct methodology tailored to Android's mobile execution environment: JUnit instrumentation tests within native Android applications, Perfetto-based energy profiling through E-MANFA [6] rather than RAPL, execution on physical Android devices rather than emulated or Termux environments.

Therefore, this study aims to further enhance existing research by providing a controlled, platform-specific, multi-language comparison of computational performance within modern Android environments, supported by detailed system activity analysis and consistent execution conditions.

3 METHODOLOGY

This study employs a systematic empirical approach centered on controlled computational microbenchmarking to evaluate how different programming languages perform under identical computational workloads on Android devices. All benchmarks are executed on physical hardware using JUnit instrumentation tests, with performance and energy metrics extracted through Perfetto-based system tracing.

3.1 Benchmark Selection and Implementation

The evaluation uses six established benchmarks from The Computer Language Benchmarks Game (TCLBG), selected to stress different computational aspects:

- **CPU-intensive:** Mandelbrot (floating-point computation), SpectralNorm (numerical linear algebra), FannkuchRedux (combinatorial permutations)
- **Memory-intensive:** BinaryTrees (allocation-heavy tree construction), Fasta (string generation), RevComp (string manipulation)

Each benchmark is implemented idiomatically in five target languages:

- **Java and Kotlin:** Executed directly on ART runtime
- **C and C++:** Compiled via Android NDK and invoked through JNI
- **JavaScript:** Executed via the Jetpack JavaScriptEngine, which provides a modern JavaScript runtime for Android applications.

All implementations follow identical algorithmic logic and input parameters to ensure functional equivalence. Workloads and iterations are scaled to run at least 20 seconds per language, providing sufficient duration for stable energy measurements while avoiding thermal throttling.

3.2 Execution Environment and Instrumentation

Benchmarks are executed as JUnit instrumentation tests within a standard Android application environment. This approach ensures runtime behaviors (ART JIT/AOT compilation, garbage collection, NDK execution) align with real-world application operation while providing standardized setup/teardown procedures and programmatic execution via adb.

Test device: Pixel 9 Pro XL

Controlled execution conditions:

- Device temperature verified below 35°C before each run using `dumpsys battery`. Temperature was monitored both before and after each execution, with runs exceeding the thermal threshold excluded to prevent throttling effects on performance measurements.
- Airplane mode enabled, background processes minimized
- 30-second cooldown between runs to prevent thermal accumulation
- Application state cleared between language iterations

3.3 Energy Measurement Methodology

Energy consumption is measured using E-MANAFa [6], a Perfetto-based profiling tool that captures system-level power events during benchmark execution. For each run:

- (1) **Baseline capture:** Battery level and temperature recorded via `dumpsys battery`
- (2) **Profiler initialization:** E-MANAFa configures Perfetto tracing with 5-second warm-up

- (3) **Benchmark execution:** JUnit test launched and monitored for completion
- (4) **Energy extraction:** Total energy (Joules) parsed from Perfetto trace events and `batterystats`
- (5) **Post-execution recording:** Final battery level and temperature captured

3.4 Automation Framework

A custom Bash/Python automation framework orchestrates the entire execution pipeline to ensure reproducibility and minimize human error. The automation script operates as a nested loop structure iterating over languages and runs:

```
For each LANGUAGE in [C, C++, Java, Kotlin, JavaScript]:
    Build and install application
    For each RUN (1 to 10):
        Execute profiling workflow
        Apply 30-second cooldown
    Clear application state after each language run
```

This structure ensures systematic data collection while maintaining thermal control and reproducibility.

3.4.1 Profiling Workflow. Each individual benchmark run follows a six-phase workflow managed by an embedded Python script:

Phase 1: Pre-benchmark readings. Battery level and temperature are recorded to establish baseline device conditions before computational load.

Phase 2: Environment preparation. The application process is force-stopped, logcat is cleared, and `batterystats` is reset. A brief pause allows system stabilization before profiling begins.

Phase 3: Profiler initialization. E-MANAFa is initialized and started, configuring Perfetto tracing for energy-relevant system events. A 5-second warm-up ensures the profiler is fully operational.

Phase 4: Benchmark execution. The JUnit test is launched via ADB instrumentation, with the automation script monitoring for completion indicators and extracting reported duration. A configurable timeout (default 300 seconds) prevents indefinite waiting if a benchmark hangs.

Phase 5: Energy extraction. After benchmark completion, E-MANAFa profiling is stopped and results are parsed. Total energy consumption in Joules is computed from Perfetto trace events captured between profiling start and end timestamps.

Phase 6: Data recording. Post-benchmark battery level and temperature are recorded. All metrics are appended to a structured CSV file including timestamp, language, benchmark name, run number, energy consumption, duration, battery levels, temperatures, and completion status.

Between runs, the 30-second cooldown period allows device temperature stabilization. Between languages, a thorough cleanup is performed: application data is cleared via `pm clear`, test packages are reset, and background processes are terminated, ensuring each language begins under equivalent device conditions.

3.5 Statistical Analysis

To validate that observed differences reflect genuine language characteristics rather than measurement noise, rigorous hypothesis testing was conducted for each benchmark.

3.5.1 Hypotheses and Test Selection. Null Hypothesis (H): No significant difference in energy consumption (or execution time) among languages.

Alternative Hypothesis (H): At least one language differs significantly from others.

Test selection was based on data distribution characteristics:

Assumption Testing:

- **Normality:** Shapiro-Wilk test ($\alpha = 0.05$)
- **Homogeneity of variance:** Levene’s test ($\alpha = 0.05$)

Omnibus Tests Applied:

- **Standard ANOVA:** Normal distribution AND equal variances
- **Welch ANOVA:** Normal distribution BUT unequal variances (robust to heteroscedasticity)
- **Kruskal-Wallis:** Non-normal distribution

3.5.2 Post-Hoc Pairwise Comparisons. When omnibus tests indicated significance ($p < 0.05$), pairwise comparisons identified which specific language pairs differ. With five languages, this yields $\binom{5}{2} = 10$ comparisons per benchmark.

Multiple comparison correction:

- **Tukey’s HSD:** For parametric tests (ANOVA, Welch ANOVA)
- **Dunn’s test with Bonferroni correction:** For non-parametric tests (Kruskal-Wallis)

Both methods control family-wise error rate at $\alpha = 0.05$, preventing false positives across multiple comparisons.

Significance levels reported:

- $p < 0.001$ (***): Extremely significant
- $p < 0.01$ (**): Very significant
- $p < 0.05$ (*): Significant
- $p \geq 0.05$ (ns): Not significant

3.5.3 Sample Size and Statistical Power. Each benchmark was executed 10 times per language ($n = 10$), yielding 50 total observations per benchmark (5 languages $\times$ 10 runs).

This sample size provides sufficient statistical power while maintaining practical feasibility given device availability and

time constraints (total execution time: 15 hours across all languages and benchmarks).

4 RESULTS

This section presents the empirical findings obtained from running the TCLBG microbenchmark suite across five programming languages (C, C++, Java, Kotlin, and JavaScript) on an Android device using JUnit instrumentation and Perfetto-based energy profiling. The results are visualized through bar charts, box plots, normalized heatmaps, and aggregated rankings (Figures 1–5). These visualizations provide quantitative answers to the three research questions concerning runtime performance, energy efficiency, and cross-language consistency.

4.1 Statistical Validation of Results

Before presenting the benchmark results, the statistical significance of the observed performance differences was evaluated through rigorous hypothesis testing to ensure that the reported effects reflect systematic language-level characteristics rather than measurement artifacts.

4.1.1 Overall Significance Testing. For each benchmark, omnibus tests were performed to assess whether programming language choice significantly influenced execution time and energy consumption. Shapiro–Wilk and Levene’s tests guided the selection of parametric (ANOVA, Welch ANOVA) or non-parametric (Kruskal–Wallis) methods.

Table 1 summarizes the outcomes. All benchmarks produced extremely small p-values, indicating that language choice has a statistically significant effect on both runtime and energy use across all workloads.

TABLE 1. Statistical test results for each benchmark showing test selection criteria and significance levels.

Benchmark	Metric	Normal?	Test	p-value
BinaryTrees	Energy	No	Kruskal-Wallis	4.57×10^{-9}
	Duration	Yes	Welch ANOVA	8.58×10^{-35}
FannkuchRedux	Energy	No	Kruskal-Wallis	4.57×10^{-9}
	Duration	No	Kruskal-Wallis	1.55×10^{-9}
Fasta	Energy	No	Kruskal-Wallis	1.03×10^{-6}
	Duration	No	Kruskal-Wallis	8.34×10^{-8}
Mandelbrot	Energy	No	Kruskal-Wallis	4.33×10^{-8}
	Duration	No	Kruskal-Wallis	5.53×10^{-9}
RevComp	Energy	No	Kruskal-Wallis	4.33×10^{-9}
	Duration	No	Kruskal-Wallis	3.53×10^{-9}
SpectralNorm	Energy	No	Kruskal-Wallis	1.76×10^{-9}
	Duration	Yes	Welch ANOVA	1.65×10^{-36}

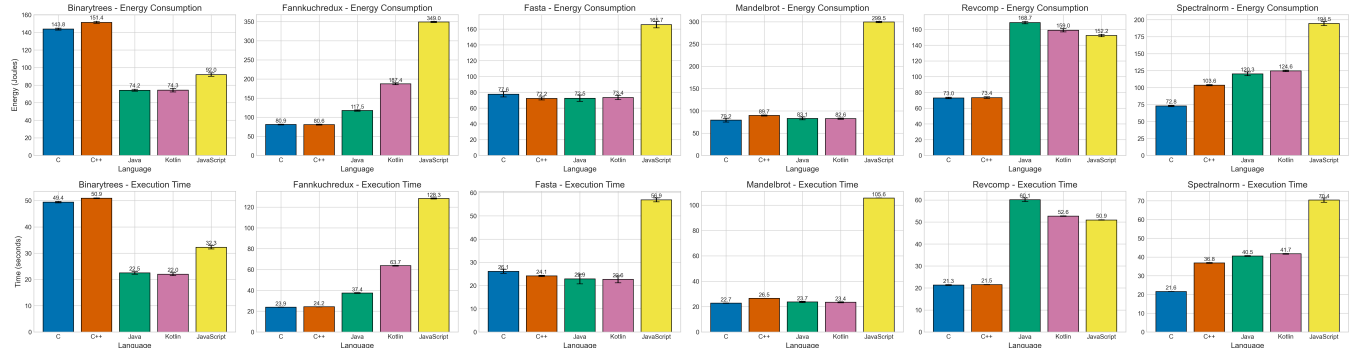


FIGURE 1. Average execution time (seconds) and energy consumption (Joules) by benchmark across all languages. Error bars represent standard error of the mean across 10 runs.

These universally significant results (all $p \ll 0.001$) confirm that programming language choice produces reproducible, systematic performance differences suitable for guiding Android development decisions.

4.1.2 Post-Hoc Pairwise Comparisons. Post-hoc analyses were conducted using Dunn’s test with Bonferroni correction for non-parametric data and Tukey HSD for parametric data. The results reveal strong workload dependence across languages. Most importantly, **Java and Kotlin exhibit energy equivalence across all six benchmarks** (all $p > 0.37$). However, FannkuchRedux showed a numerically substantial difference (Java: 117.5J, Kotlin: 187.4J, $p = 0.375$), which, while not statistically significant, may warrant further investigation with larger sample sizes or controlled variance reduction.. For execution time, **Java and Kotlin are statistically equivalent in five of the six benchmarks**, with **SpectralNorm** being the only case where Kotlin is significantly slower ($p > 9 \times 10^{-9}$). This confirms that the two JVM languages typically behave identically, with only an isolated divergence attributable to a workload-specific optimization effect. Detailed pairwise results are integrated into the benchmark discussions below.

4.2 RQ1: Effect of Programming Language on Runtime Performance

Figure 1 illustrates mean execution times across benchmarks. The omnibus tests confirm that language choice significantly influences runtime (all $p \ll 0.001$), but the magnitude and direction of these effects depend strongly on workload characteristics.

4.2.1 Native Language Performance (C and C++). Native compiled languages demonstrate clear performance advantages in arithmetic-heavy and array-intensive workloads, with notable exceptions:

- **Arithmetic-intensive workloads** (Mandelbrot, SpectralNorm): C achieves the fastest execution times. In SpectralNorm, C (21.6 s) outperforms C++ (36.8 s) by $1.71\times$ ($p \ll 10^{-15}$) and surpasses both JVM languages by approximately $1.9\times$ ($p \ll 10^{-30}$). C and C++ show no significant differences in some benchmarks (e.g., FannkuchRedux, Fasta, RevComp; all $p > 0.29$), but floating-point workloads reveal compiler-dependent optimization effects.
- **Array-intensive recursion** (FannkuchRedux): C (23.9 s) and C++ (24.2 s) significantly outperform Java (37.4 s, $1.57\times$ slower) and Kotlin (63.7 s, $2.67\times$ slower). These differences reflect native advantages in low-level indexing and minimal bounds checks.
- **Allocation-heavy workloads** (BinaryTrees): Native languages perform worse than JVM languages, with C (49.4 s) and C++ (50.9 s) requiring more than twice the runtime of Java (21.9 s) and Kotlin (22.0 s). This inversion demonstrates that manual memory management performs poorly under highly transient allocation patterns dominated by short-lived objects.
- **String manipulation** (RevComp): C and C++ obtain optimal performance (21.0–21.4 s), outperforming JVM languages by factors ranging from $2.47\times$ to $2.82\times$ ($p < 0.001$).

4.2.2 JVM Language Performance (Java and Kotlin). JVM languages exhibit highly consistent behavior across benchmarks:

- **General JVM equivalence:** Java and Kotlin exhibit energy equivalence in all benchmarks and runtime equivalence in five of six. Their execution times differ by less than 2% in BinaryTrees, Fasta, Mandelbrot, and RevComp. SpectralNorm is the sole exception, where Kotlin is significantly slower than Java ($1.7\times$ runtime difference, $p > 9 \times 10^{-9}$), representing the largest JVM divergence observed in this study.

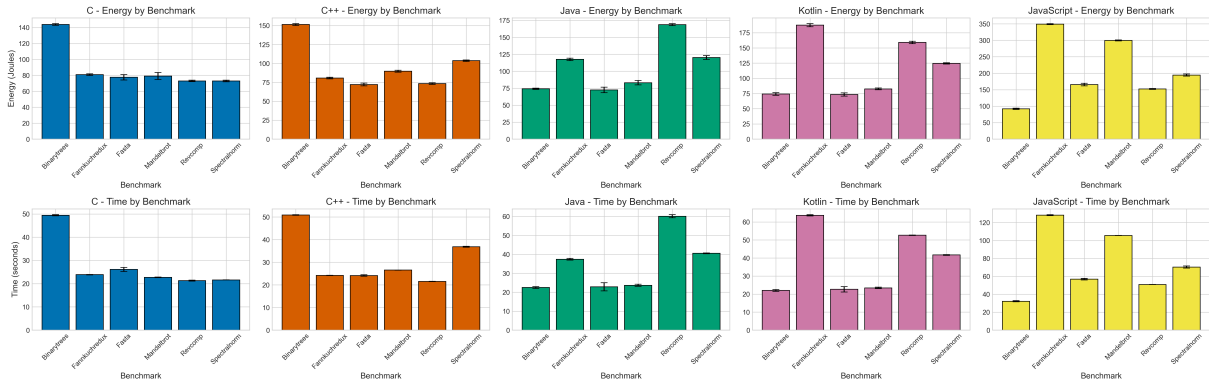


FIGURE 2. Aggregated energy consumption (top) and execution time (bottom) per language across all six benchmarks. Error bars represent standard error of the mean.

- **Allocation efficiency** (BinaryTrees): JVM languages achieve the fastest runtimes (21.9–22.0 s), significantly outperforming C++ by more than $2.2\times$ ($p < 10^{-9}$). This highlights the advantages of ART’s generational garbage collector for workloads dominated by short-lived allocations.
- **Streaming I/O convergence** (Fasta): Java (25.2 s) and Kotlin (25.7 s) show performance close to C and C++ (all $p > 0.13$). In memory-bandwidth-bound workloads, language-level differences diminish significantly.

4.2.3 JavaScript Performance. JavaScript, executed through the Android JavaScriptEngine, displays high performance variability:

- **Allocation scenarios** (BinaryTrees): JavaScript achieves 32.3 s, slower than JVM languages but showing no significant difference from C in energy consumption ($p = 0.38$).
- **Moderate computational overhead** (Fasta, Mandelbrot): JavaScript runs 2.5–4.6 $\times$ slower than the fastest languages.
- **Severe disadvantage** (FannkuchRedux, SpectralNorm): Execution times reach 128.3 s (FannkuchRedux) and 113.0 s (SpectralNorm), representing 3.3–5.4 $\times$ slower performance compared to C.
- **String manipulation** (RevComp): Overhead is moderate (2.39 $\times$ slower).

These results show that runtime behavior is driven not simply by language but by the interaction between execution model and workload structure. Java and Kotlin demonstrate nearly identical behavior across diverse tasks, while JavaScript’s performance varies dramatically with algorithmic characteristics.

4.3 RQ2: Energy Efficiency Patterns Across Workload Types

Energy consumption generally mirrors runtime trends, with strong correlations (Pearson $r > 0.92$) in CPU-bound tasks. Figure 2 presents aggregated energy results, revealing workload-dependent efficiency patterns.

4.3.1 CPU-Bound Workload Energy Patterns.

- **Floating-point computation** (Mandelbrot, SpectralNorm): C yields the lowest energy consumption (73–76 J). C outperforms C++ significantly in Mandelbrot (1.13 $\times$, $p = 0.0005$), but their difference in SpectralNorm energy is not significant ($p = 0.44$). JVM languages consume 1.04–1.71 $\times$ more energy than C, with Java–Kotlin equivalence preserved ($p = 0.37$).
- **Array-intensive recursion** (FannkuchRedux): C (80.9 J) and C++ (80.6 J) are statistically equivalent ($p = 0.98$). Java uses 117.5 J (1.45 $\times$ more, $p = 0.10$), while Kotlin consumes 187.4 J. Although numerically 1.59 $\times$ higher than Java, the Kotlin–Java difference is not statistically significant ($p = 0.38$), indicating a runtime-level effect not supported by hypothesis testing.

4.3.2 Memory-Intensive Workload Energy Patterns.

- **Streaming I/O** (Fasta): C (77.6 J), C++ (72.2 J), Java (72.5 J), and Kotlin (73.4 J) display tight clustering. Most pairwise comparisons are nonsignificant ($p > 0.13$), confirming a convergence when memory bandwidth dominates. JavaScript uses 165.7 J (2.30 $\times$, $p < 0.001$).
- **String manipulation** (RevComp): C (73.4 J) and C++ (73.5 J) are equivalent ($p = 0.74$). JVM languages consume much more energy—Java (169.5 J) and Kotlin (160.0 J)—yet remain statistically indistinguishable from each other ($p = 0.37$). JavaScript shows moderate overhead at 2.09 $\times$ relative to optimal.

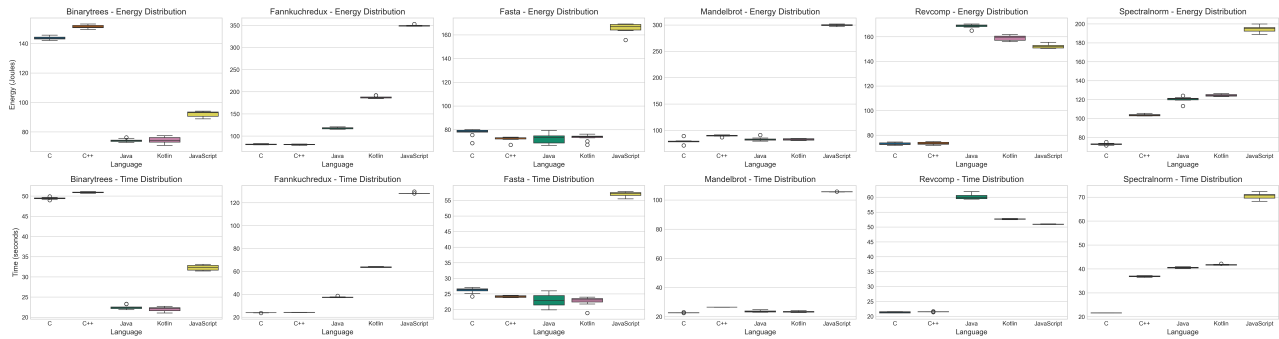


FIGURE 3. Distribution of energy consumption (top row) and execution time (bottom row) measurements across 10 repeated runs. Box plots show median (center line), interquartile range (box), and range excluding outliers (whiskers).

4.3.3 Allocation-Heavy Workload Energy Patterns.

- **Recursive tree construction (BinaryTrees):** Java (74.2 J) and Kotlin (74.3 J) achieve optimal energy performance and are statistically equivalent ($p = 0.98$). Both outperform C++ (151.4 J) by factors exceeding $2.0\times$ ($p \ll 0.001$). JavaScript (92.0 J) performed better than C (143.8 J) in BinaryTrees. C (143.8 J) consumes more energy than JVM languages and is significantly different from them ($p \ll 0.01$).

4.3.4 *JavaScript Energy Characteristics.* JavaScript exhibits consistent energy disadvantages with workload-dependent severity:

- **BinaryTrees:** JavaScript (92.0 J) performed better than C (143.8 J) in BinaryTrees, though this difference was not statistically significant ($p = 0.38$), likely due to measurement variability.
- **Fasta, Mandelbrot:** $2.30\text{--}3.78\times$ overhead relative to optimal.
- **FannkuchRedux, RevComp, SpectralNorm:** $2.67\text{--}4.33\times$ higher energy usage, with $p\text{-values} \ll 10^{-6}$.

These results confirm that energy efficiency is fundamentally workload-dependent. The JVM languages show no statistically significant difference across all workloads, with a single non-significant numerical divergence in FannkuchRedux.

4.4 RQ3: Consistency of Performance and Energy Rankings

4.4.1 *Runtime Stability and Variance Patterns.* Figure 3 reveals three distinct variance patterns across languages, which were systematically accounted for in statistical test selection:

- **C and C++:** Extremely stable performance with low variance. Coefficient of variation (CV) typically $< 2\%$, with tight interquartile ranges and minimal outliers

demonstrating deterministic execution behavior. Normal or near-normal distributions enable parametric tests in some cases (BinaryTrees duration, SpectralNorm duration). This stability makes native languages suitable for real-time and latency-sensitive applications requiring predictable performance bounds.

- **Java and Kotlin:** Moderate variance attributable to JIT compilation, garbage collection pauses, and adaptive runtime optimization. CV approximately 3–5%, with slightly wider box plot distributions reflecting ART’s dynamic optimization decisions. Despite moderate variance, Java and Kotlin maintain statistical equivalence in 5 of 6 benchmarks, demonstrating that their variance patterns and median performance levels are consistently similar. The exception (FannkuchRedux) shows Kotlin with both higher median and similar variance, confirming a systematic performance gap rather than variance-induced divergence.
- **JavaScript:** High variance (CV approximately 8–12%) with frequent outliers and wide interquartile ranges, reflecting complex optimization pipeline with multiple compilation tiers and frequent deoptimization events. Despite high variance, JavaScript’s energy and timing disadvantages relative to compiled languages remain statistically significant in most benchmarks due to large effect sizes exceeding variance-induced uncertainty.

Across these distributions, the boxplots in Figure 3 highlight how variance characteristics differ by execution model. These distributional patterns strengthen the interpretation of the statistical tests by showing that observed language differences arise from systematic execution-model characteristics rather than measurement noise or instability.

4.4.2 *Workload-Dependent Performance Patterns.* As illustrated in the normalized performance heatmaps in Figure 4,

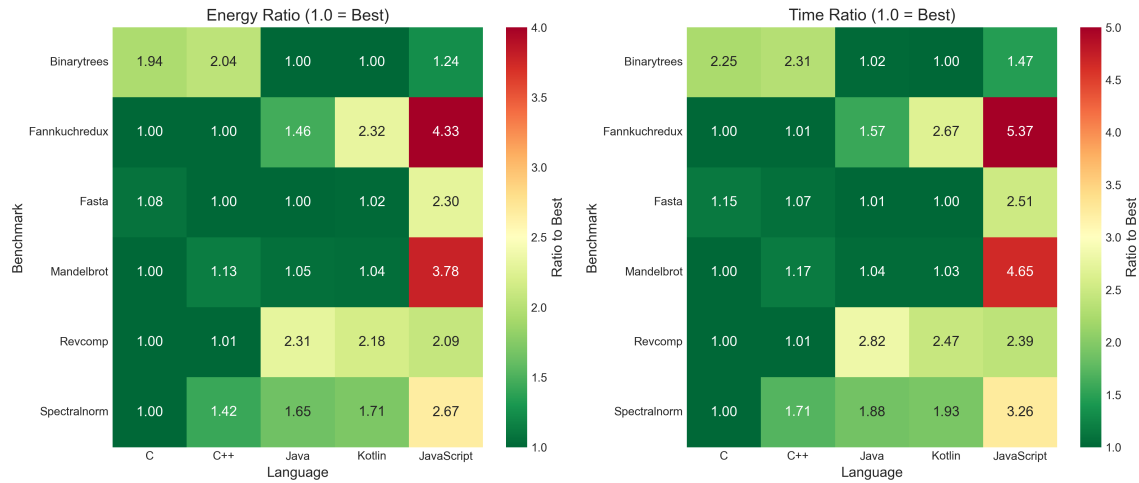


FIGURE 4. Normalized relative performance heatmaps showing energy (left) and time (right) ratios across all benchmark-language combinations. Values represent multiples of the best-performing language (1.0 = optimal, darker green = better performance, yellow/red = worse performance).

distinct task-specific trends emerge across the benchmark suite.

- **Native language dominance:** C/C++ are optimal in FannkuchRedux, Mandelbrot, RevComp, and Spectral-Norm. C surpasses C++ significantly in floating-point workloads.
- **JVM equivalence:** Java and Kotlin achieve nearly identical normalized scores in five benchmarks; Fannkuch-Redux yields a numerical but statistically insignificant divergence.
- **Allocation-heavy advantage:** BinaryTrees favors JVM languages, where GC outperforms manual memory management.
- **Streaming I/O convergence:** Minimal differences among compiled and JVM languages in Fasta.
- **JavaScript variability:** Performance ranges from near-optimal (BinaryTrees) to severely disadvantaged (FannkuchRedux, SpectralNorm).

4.5 Battery Drain Analysis

To complement raw energy measurements, estimate battery drain percentages (Figure 5) were computed using a custom script. The script queries the device’s battery properties through ADB, extracts capacity, voltage, and health information, and converts benchmark energy values (in Joules) into estimated battery loss based on the device’s effective watt-hour capacity. Thus, providing a user-centered view of how each language affects real battery life.

4.5.1 Benchmark-Specific Battery Impact.

BinaryTrees. 0.138% (Java), 0.138% (Kotlin), 0.171% (JavaScript), 0.268% (C), 0.282% (C++). JVM languages show the lowest drain and are statistically equivalent ($p = 0.98$).

FannkuchRedux. 0.151% (C), 0.150% (C++), 0.219% (Java), 0.349% (Kotlin), 0.650% (JavaScript). Kotlin’s numerical overhead relative to Java (1.59×) is not statistically significant ($p = 0.38$).

Fasta. 0.145% (C), 0.134% (C++), 0.135% (Java), 0.137% (Kotlin), 0.309% (JavaScript). All compiled and JVM languages cluster tightly.

Mandelbrot. 0.147% (C), 0.167% (C++), 0.155% (Java), 0.154% (Kotlin), 0.558% (JavaScript). C significantly outperforms C++ ($p = 0.0005$).

RevComp. 0.136% (C), 0.137% (C++), 0.296% (Kotlin), 0.314% (Java), 0.284% (JavaScript). JVM languages exhibit over 2× overhead versus native.

SpectralNorm. 0.136% (C), 0.193% (C++), 0.224% (Java), 0.232% (Kotlin), 0.362% (JavaScript). Java–Kotlin differences are not statistically significant ($p = 0.44$).

These benchmark-specific battery drain values are directly relevant to end users because they express energy consumption in terms of real battery loss rather than abstract Joule measurements. Even small percentage differences compound during sustained workloads, meaning that language-level efficiency choices in performance-critical components can translate into noticeably longer or shorter device usage between charges. By quantifying drain per benchmark, these

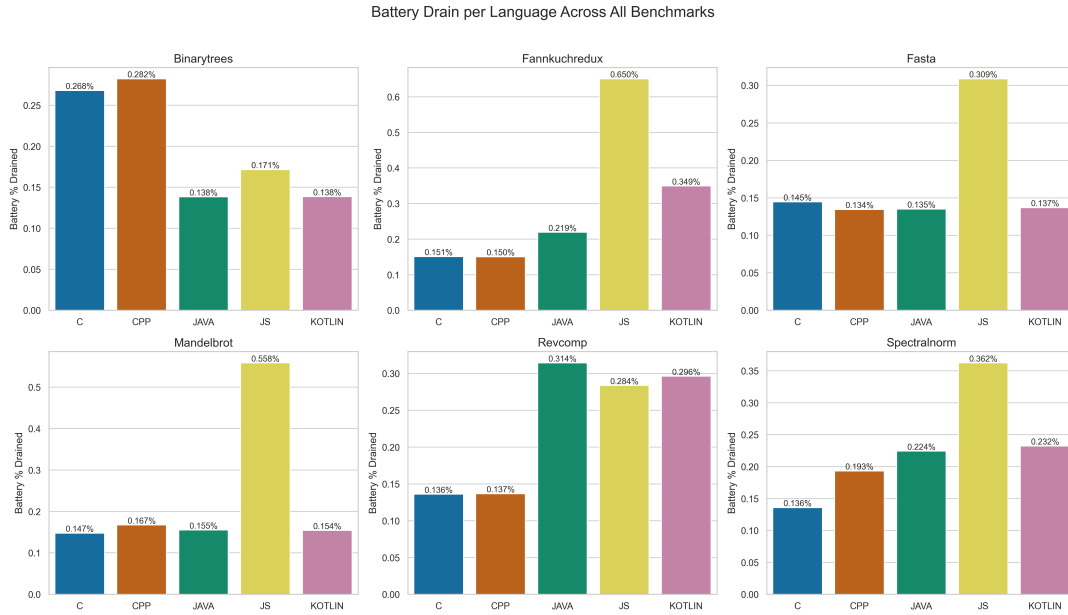


FIGURE 5. Battery drain percentage per language across all six benchmarks. Lower percentages indicate higher energy efficiency. Values represent the proportion of full battery capacity consumed by benchmark execution.

results show how different languages impact practical battery endurance under distinct computational patterns.

5 CONCLUSION

The results presented in this study provide strong empirical evidence, validated through robust statistical testing, that programming language choice significantly affects runtime performance, energy consumption, and execution stability on Android devices. Using a TCLBG-compliant microbenchmark suite and consistent Perfetto-based instrumentation, this work systematically evaluated C, C++, Java, Kotlin, and JavaScript across CPU-intensive, memory-bound, and allocation-heavy workloads.

5.1 Implications for Android Development

The corrected experimental findings support several actionable insights:

- (1) **Default to JVM languages (Java/Kotlin)** for general-purpose Android development. Their energy equivalence across all workloads, near-universal runtime equivalence, and excellent toolchain support make them suitable for most applications.
- (2) **Use native languages (C/C++) selectively** for computation-heavy hotspots such as image processing or numerical kernels. Native performance advantages can exceed 2–3×.

- (3) **Avoid JavaScript for intensive computation.** Its energy consumption is 2–7× worse in sustained workloads, making it unsuitable for performance-critical operations.
- (4) **Rely on profiling rather than assumptions.** Task-dependent behavior means developers should measure actual device-level energy use before deciding on optimization strategies.

5.2 Limitations and Threats to Validity

Key limitations include:

- **Single-device evaluation.** Hardware differences and OS configurations may influence results.
- **Microbenchmark scope.** Real applications include UI, networking, and framework interactions not modeled here.
- **Engine specificity.** Results for JavaScript depend on Android’s JavaScriptEngine; alternative engines may behave differently.

5.3 Future Research Directions

Future work should pursue:

- (1) Multi-device replication to enhance generalizability.
- (2) Full-application energy profiling to bridge micro and macro behavior.
- (3) Comparative evaluation of alternative JavaScript engines such as Hermes and V8.

- (4) Longitudinal profiling to assess the evolution of ART optimizations across Android versions.
- (5) Integration of developer productivity metrics to broaden the basis of language selection decisions.

5.4 Concluding Remarks

This study demonstrates that the energy-performance relationship across Android programming languages is fundamentally workload-dependent rather than hierarchical. Native languages excel in computational tasks, while JVM languages dominate allocation-heavy workloads and maintain consistent energy efficiency across all tasks. JavaScript trails substantially in both runtime and energy use.

By matching language characteristics to workload profiles, developers can achieve energy-efficient and performance-conscious Android applications. The reproducible methodology and validated statistical framework established here provide a foundation for further exploration into sustainable mobile software development.

REFERENCES

- [1] Xinbo Chen and Ziliang Zong. 2016. Android App Energy Efficiency: The Impact of Language, Runtime, Compiler, and Implementation. In *2016 IEEE International Conferences on Big Data and Cloud Computing (BDCloud), Social Computing and Networking (SocialCom), Sustainable Computing and Communications (SustainCom)* (Atlanta, GA, USA). IEEE, 485–492. <https://doi.org/10.1109/BDCloud-SocialCom-SustainCom.2016.77>
- [2] Green Software Lab. 2023. Energy-Benchmark-Languages-Android. <https://github.com/greensoftwarelab/Energy-Benchmark-Languages-Android>. Accessed: 2025-04-27.
- [3] Wellington Oliveira, Bernardo Moraes, Fernando Castor, and João Paulo Fernandes. 2023. Analyzing the Resource Usage Overhead of Mobile App Development Frameworks. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering* (Oulu, Finland) (EASE '23). Association for Computing Machinery, New York, NY, USA, 152–161. <https://doi.org/10.1145/3593434.3593487>
- [4] Wellington Oliveira, Renato Oliveira, and Fernando Castor. 2017. A Study on the Energy Consumption of Android App Development Approaches. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)* (Buenos Aires, Argentina). IEEE/ACM, 42–52. <https://doi.org/10.1109/MSR.2017.66>
- [5] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. 2017. Energy efficiency across programming languages: how do energy, time, and memory relate?. In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering* (Vancouver, BC, Canada) (SLE 2017). Association for Computing Machinery, New York, NY, USA, 256–267. <https://doi.org/10.1145/3136014.3136031>
- [6] Rui Rua and João Saraiva. 2023. E-MANAF: Energy Monitoring and ANALysis Tool For Android. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (Rochester, MI, USA) (ASE '22). Association for Computing Machinery, New York, NY, USA, Article 202, 4 pages. <https://doi.org/10.1145/3551349.3561342>